# Quick Start

*Build a working agent and deploy it — start to finish. Aimed at someone new to the SDK; basic command-line comfort is all you need.*

By the end you'll have scaffolded an agent project, customised it, run it locally, and deployed it to AWS. The example throughout is a small **AG-UI** (chat) agent called **Data Analyst** that can analyse data files.

**Before you start, you'll need:**

- **Python 3.12 or 3.13** installed (`python3 --version`).
- **Access to the GitLab package registry** where the SDK is published (Step 1 covers this).
- For the deploy step: a GitLab project with CI set up, and an AWS account prepared for agent deployment — see Deployment and AWS Deployment Role. If you only want to try the SDK locally, you can stop after Step 5.

## Step 1 — Get access to the SDK package

The SDK is published to an internal **GitLab package registry**, not the public internet. You install it by pointing `pip` at that registry with an access token.

If you have the GitLab CLI (`glab`) set up, you can use your existing session token:

```
# Turn your glab session into a token pip can use
GL_TOKEN=$(python3 -c 'import urllib.parse,subprocess;
print(urllib.parse.quote(subprocess.check_output(["glab","auth","token"]).decode().strip(),
safe=""))')
```

If you don't use `glab`, create a GitLab Personal Access Token with the `read_api` scope and use it in place of the token above. Your platform team can point you to the registry.

# Step 2 — Install the SDK tools

Create a fresh virtual environment and install the SDK with its command-line tools (the `[cli]` extra):

```
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip

pip install "ai-foundry-platform-sdk[cli]" \
  --extra-index-url
"https://oauth2:${GL_TOKEN}@gitlab.com/api/v4/projects/80473976/packages/pypi/simple"

ai-foundry --version    # confirms the install, e.g. "ai-foundry, version 1.2.0"
```

That gives you the `ai-foundry` command used in the next steps.

# Step 3 — Set up your project (`ai-foundry init`)

`ai-foundry init` creates the **repository chassis** — the shared structure that one or more agents live in. Run it once per project:

```
ai-foundry init --name my-agents --aws-region eu-central-1
```

It asks a short series of questions about your environments (your AWS account ID, the Terraform state bucket, your network settings, and the login audiences). These are collected once for non-production, then for production, and saved so you don't have to repeat them per agent. Your platform/cloud team can give you the right values; leave a prompt blank to fill it in later.

This creates the project skeleton (a `.ai-foundry.json` config, the CI pipeline file, and an empty `agents/` folder). **No cloud resources are created yet** — `init` only writes files.

# Step 4 — Create your agent (`ai-foundry new-agent`)

Now add an agent. Because this one serves a chat UI, use the **AG-UI** protocol:

```
ai-foundry new-agent \
  --name "Data Analyst" \
  --description "Analyses CSV and Excel files: charts and summaries via a code sandbox." \
  --protocol agui \
  --with-tools
```

This scaffolds a complete, self-contained agent under `agents/data_analyst/` — application code (`app.py`), a starter tool file, a `Dockerfile`, tests, its own cloud setup (`infra/`), and a CI entry. You now have a runnable agent project.

**Choosing a protocol:** use `--protocol agui` for agents people chat with through a UI, and `--protocol a2a` for backend agents called by other agents or services. See Building Your Agent.

# Step 5 — Make it yours, and run it locally

Open `agents/data_analyst/app.py`. The whole agent is defined in one place. You mainly change two things: the **instructions** (the domain prompt) and the **tools**. Here's the shape:

```
from pathlib import Path
import os

from ai_foundry_platform_sdk.agent import AgentDefinition, AuthConfig, Skill, SkillPackage
from ai_foundry_platform_sdk.app import create_agui_app
from ai_foundry_platform_sdk.tools.code_interpreter import code_interpreter_tool
from ai_foundry_platform_sdk.tools.files import files_tool

DOMAIN_PROMPT = """\
You are Data Analyst, a data-exploration assistant. Help the user analyse
tabular data (CSVs, Excel sheets) using a Python code sandbox. Save any
charts to outputs/ so they come back as download links.
"""
```

```
agent = AgentDefinition(
    name="Data Analyst",
    description="Analyses CSVs/Excel: charts and summaries via a code sandbox.",
    domain_prompt=DOMAIN_PROMPT,              # your instructions
    tools=[                                   # what the agent can do
        code_interpreter_tool(),              # run Python safely
        files_tool(),                         # read files the user attaches
    ],
    skills=[
        Skill(
            id="data-analysis",
            name="Data Analysis & Visualisation",
            description="Analyse CSVs/Excel and generate charts.",
            tags=["data", "analysis"],
        ),
    ],
    auth=AuthConfig(                          # who can use it (see Step 6)
        allowed_groups=[g for g in os.environ.get("ALLOWED_AD_GROUPS", "").split(",") if g],
    ),
)

app = create_agui_app(agent)
```

Edit `DOMAIN_PROMPT` to describe your agent, and adjust the `tools` list. To run it on your machine:

```
cd agents/data_analyst
# create a .env from the provided .env.example and set a model key for local dev
uvicorn app:app --host 0.0.0.0 --port 8080
```

For local runs you can set a model key directly and skip access control — see Building Your Agent and Security & Access. More on tools is in Tools & Files.

# Step 6 — Deploy it

Deployment is driven by your GitLab CI pipeline. In short:

1. **Commit and push your project to your GitLab repository.** The pipeline runs automatically (deploys to `dev` on merge to `main`).
2. The pipeline **deploys first, then builds**: it provisions your agent's AWS resources (container registry, runtime, secret store, logs), then builds and pushes your agent's container image. The deployed agent launches via `ai-foundry-serve` (generated for you), which loads your keys from the agent secret at startup.
3. After the first successful deploy, **two one-time setup steps**:
   - **Add your keys.** Deployment creates one empty **agent secret** named like `my-agents-dev-data_analyst-secret`. Put your model key (and any telemetry keys) into it as JSON — `{"PORTKEY_API_KEY":"pk-..."}`, optionally with `DD_API_KEY` / `AMPLITUDE_API_KEY` / `CONFIDENT_API_KEY`. The `new-agent` command prints the exact command to run.
   - **Allow the right people in.** Set `ALLOWED_AD_GROUPS` (the company groups allowed to use the agent) in the agent's settings and re-deploy. Until you do, human users are denied by default. See Security & Access.

This step assumes your AWS account is prepared to host agents. The full picture — what gets created, the settings you can change, and the AWS permissions an account needs — is in **Deployment** and **AWS Deployment Role**.

# Step 7 — Try your agent

Once deployed, your agent runs on **AWS AgentCore** and exposes a secure endpoint (AG-UI for chat agents, A2A for service agents), protected by company login.

You can exercise it in whatever way suits your team, for example:

- through any **AG-UI-compatible chat frontend**, or a frontend your team deploys;
- via the **AWS AgentCore console / agent sandbox**;
- with **direct authenticated API requests** (sending a valid company login token).

Send it a couple of real tasks to confirm it's working end to end — for the Data Analyst example, something like *"summarise this CSV"* with a file attached, or *"plot $y = x^2$ for x from -5 to 5"*.

# Where next

| I want to... | Read |
| --- | --- |
| Understand the agent definition, prompts, and models in depth | Building Your Agent |
| Add more tools, files, or a database | Tools & Files |
| Let the agent remember conversations | Memory |
| Control who can use the agent | Security & Access |
| Understand the deploy pipeline and AWS resources | Deployment |
| Prepare an AWS account for deployment | AWS Deployment Role |
| Get stuck? | FAQ & Troubleshooting |